New Generation Data Models and DBMSs Course - Assignment 3 - NoSQL Queries

Carlo Falchi (Matr. 67340A)[1*]
and Francesco Palladini (Matr. 85652A)[1*]

[1]Department of Computer Science, University of Milan, Italy.

*Corresponding author(s). E-mail(s): carlo.falchi@studenti.unimi.it;
francesco.palladini@studenti.unimi.it;

1 Assignment

1. For each university, identify the name, kind and city of the university that are located in California or Massachusetts along with the name, cfu, teacher surname and number of participants of the courses that it offers.
2. For each university, identify the teachers that teach the top three courses in terms of number of participants.
3. Identify the students that passed all their exams for courses with a written text with an average mark above 27.
4. For each course, report the list of students that participated to the course. Report also the number of students that passed the course with a mark between 18 and 23, between 24 and 27, and above 27. Report also the number of students that did not pass the course.
5. Identify the pair of students that are friend of a friend (i.e. a student should be associated with the friends of his friends).

1.1 First Query

For each university, identify the name, kind and city of the university that are located in California or Massachusetts along with the name, cfu, teacher surname and number of participants of the courses that it offers.

(a) First, we filter out only the universities in California or Massachusetts

(b) The document is then joined according to the "_id" to associate all the courses of each university
(c) The field courses is unwind to get the single element of the array
(d) A new field is added to calculate the number of participants for that course
(e) The group by operation is needed to group all the university's courses for each university _id

```
db.university.aggregate([
  {
    $match: {
      $or: [
        { state: "California" },
        { state: "Massachusetts" }
      ]
    }
  },
  {
    $lookup: {
      from: "course",
      localField: "_id",
      foreignField: "university",
      as: "courses"
    }
  },
  {
    $unwind: { path: "$courses" }
  },
  {
    $addFields: {
      numPartecipants: { $size: "$courses.participants" }
    }
  },
  {
    $group: {
      _id: "$_id",
      name: { $first: "$name" },
      kind: { $first: "$kind" },
      city: { $first: "$city" },
      courses: { $push: "$courses" }
    }
  }
])
```

1.2 Second Query

For each university, identify the teachers that teach the top three courses in terms of number of participants.

(a) First, we add a field that counts the number of participants for every course
(b) Then we group by University and Teacher Name and Surname, counting for each one the course with the max number of participants
(c) Sort the documents list in descending order
(d) We group again by university to limit to the first 3 teacher for each university
(e) The document is joined according to the _id field which is the university id, to associate the university info
(f) We got an array with one element so we unwind it
(g) Finally, we show only the requested fields: University Name and the related Teachers

```
db.course.aggregate([
  { "$addFields": { "numParticipants": { "$size": "$participants" } } },
  { "$group": { "_id": { "univ": "$university", "name": "$teacher.name", "surname": "$teacher.surname" }, "maxParticipants": { "$max": "$numParticipants" } } },
  { "$sort": { "maxParticipants": -1 } },
  { "$group": { "_id": "$_id.univ", "teachers": { "$firstN": { "input": { "name": "$_id.name", "surname": "$_id.surname" }, "n": 3 } } } },
  { "$lookup": { "from": "university", "localField": "_id", "foreignField": "_id", "as": "univ" } },
  { "$unwind": { "path": "$univ" } },
  { "$project": { "name": "$univ.name", "teachers": "$teachers" } }
])
```

1.3 Third Query

Identify the students that passed all their exams for courses with a written text with an average mark above 27.

(a) First, we do an unwind of the exams to get one for each document
(b) The document is joined with the related course for each exam passed by the student
(c) We use a match stage to filter out only the written exams
(d) We group by the student to calculate the average of the resulting exams
(e) Filter out only the students that have an average above 27
(f) We remove the unrequested fields by project stage

```
db.student.aggregate([
  { $unwind: "$exams" },
  {
    $lookup: {
      from: "course",
```

```
      localField: "exams.course",
      foreignField: "_id",
      as: "course",
      pipeline: [{ $project: { exam_type: 1 } }]
    }
  },
  { $match: { "course.exam_type": "Written" } },
  {
    $group: {
      _id: "$_id",
      average: { $avg: "$exams.mark" },
      name: { $first: "$name" },
      surname: { $first: "$surname" }
    }
  },
  { $match: { average: { $gt: 27 } } },
  { $project: { _id: 0, average: 0 } }
])
```

1.4 Fourth Query

For each course, report the list of students that participated to the course. Report also the number of students that passed the course with a mark between 18 and 23, between 24 and 27, and above 27. Report also the number of students that did not pass the course.

(a) First, we do an unwind of the participants to get an element for the document
(b) Then, we join according to the participants field in order to associate the related student info
(c) Given that we got an array of students, we do unwind to extract the only object element in the array
(d) To keep track only of exam related to the course document within the exams student array, we extract it as new fields using the $filter expression
(e) For each document, we calculate those that respect the requested range of marks.
(f) Finally, we group by course to give a sum of the requested numbers.

```
db.course.aggregate([
  { $unwind: "$participants" },
  {
    $lookup: {
      from: "student",
      localField: "participants",
      foreignField: "_id",
      as: "student"
    }
  },
  { $unwind: "$student" },
```

```
{
  $addFields: {
    currentCourseExam: {
      $first: {
        $filter: {
          input: "$student.exams",
          as: "exam",
          cond: { $eq: ["$$exam.course", "$_id"] }
        }
      }
    }
  }
},
{
  $addFields: {
    didntPassExam: {
      $cond: [{ $not: ["$currentCourseExam"] }, 1, 0]
    },
    passedwith18to23: {
      $cond: [
        {
          $and: [
            { $gte: ["$currentCourseExam.mark", 18] },
            { $lte: ["$currentCourseExam.mark", 23] }
          ]
        },
        1,
        0
      ]
    },
    passedwith24to27: {
      $cond: [
        {
          $and: [
            { $gte: ["$currentCourseExam.mark", 24] },
            { $lte: ["$currentCourseExam.mark", 27] }
          ]
        },
        1,
        0
      ]
    },
    passedabove27: {
      $cond: [{ $gt: ["$currentCourseExam.mark", 27] }, 1, 0]
    }
  }
},
{
  $group: {
    _id: "$_id",
```

```
      name: { $first: "$name" },
      participants: { $push: "$student" },
      total_passedwith18to23: { $sum: "$passedwith18to23" },
      total_passedwith24to27: { $sum: "$passedwith24to27" },
      total_passedabove27: { $sum: "$passedabove27" },
      total_didntPassExam: { $sum: "$didntPassExam" }
    }
  }
])
```

1.5 Fifth Query

Identify the pair of students that are friend of a friend (i.e. a student should be associated with the friends of his friends).

(a) First, we unwind the list of friends
(b) Then, we associate the friend info with the student collection
(c) Given that friend$_i nfo is an array, we unwind it to get the only element of the array, and then we unwind the fr$

(c) We then group by id and the friends of friends to avoid duplicates
(d) We join the student collection related to the friend of friend and unwind it
(e) Finally, we project the requested fields

```
db.student.aggregate(
[
    { $unwind: "$friends" },
    {
        $lookup: { from: "student", localField: "friends",
            foreignField: "_id", as: "friend_info" }
    },
    { $unwind: "$friend_info" },
    { $unwind: "$friend_info.friends" },
    {
        $group: {
            _id: { _id: "$_id", friendOfFriendId: "$friend_info.
                friends" },
            name: { $first: "$name" },
            surname: { $first: "$surname" }
        }
    },
    {
        $lookup: { from: "student", localField: "_id.
            friendOfFriendId", foreignField: "_id", as: "
            friendoffriend" }
    },
    { $unwind: "$friendoffriend" },
```

```
20      { $project: { _id: 0, name: 1, surname: 1, friendofFriendName
            : "$friendoffriend.name", friendofFriendSurname: "
            $friendoffriend.surname" } }
21  ])
```